

Search on Encrypted Data

Shruthi Gorantala*
University of Pennsylvania

December 15, 2014

Abstract

Over the years, cloud computing has gained popularity and outsourcing storage has gained significance. More and more information is being outsourced to store on the cloud both by industries as well as general users. However storing critical documents on the cloud does not ensure security and employing cryptographic methods such as encrypting the documents is the solution. However, encrypted document storage hinders some of the basic features such as selective retrieval of relevant documents. Search has become the most common way of accessing data and hence search over encrypted data is an active area of research. In this paper, a public key encryption scheme using bloom filters [3] has been extensively studied, implemented and evaluated in terms of performance and main areas of bottleneck are identified. To reduce the communication rate techniques from [4] are employed and a comparison of various systems is provided.

1 Introduction

Search on Encrypted data is an active area of research with the growth of cloud computing and availability of storage as a service and the need to secure documents on the cloud. Search on encrypted data was first considered by Song, Wagner and Perrig in [7]. The scheme uses pseudo random functions to employ the encrypted search but it reveals search pattern and also prone to statistical analysis when searched for too many words. Search on encrypted data can also be achieved using Secure Indexes [6] which make use of bloom filters. We consider Secure Indexes as the data structures used here (when extended) lead to a public key crypto system which allows encrypted keyword search which is resistant to statistical analysis as well as pattern leakage. the model considered in the single user settings is that Alice wants to store her documents on the Server. We consider the Server in an Honest-but-curious model. This is achieved via Secure Indexes using Bloom filters. This scheme however leaks information about search pattern and is prone to statistical analysis. When the bloom filters are extended with storage, an additively homomorphic cryptosystem can be employed to hide search pattern as well and uploading and downloading files can be achieved through oblivious modification and search respectively. The scheme was proposed in [3] However with increased security, there are quite few tradeoffs such as huge communication complexity and computations. The homomorphic properties are available in public key cryptography and hence we consider a public key setting where Bob wants to send an email to Alice through the mail server. Again the mail server is considered in the Honest-but-curious model. The scheme involves a mail box initialization step, oblivious modification of the mailbox for sending email and retrieving specific emails based on keyword search. Paillier cryptosystem is used for additively

*Advised by Brett Hemenway and Nadia Heninger

homomorphic system. We implement [3] and employ the techniques of Gentry Ramzan's scheme to reduce the communicate rate of the system. The rest of the paper is organised as follows: we provide basic preliminaries and definitions and establish notations in section 2 , Section 3 describes the schemes extensively and section 4 describes the implementation details .Section 5 reports the findings and feasibility of the scheme and Section 6 suggests further improvements according to the findings.

2 Definitions

1. Bloom Filters: A bloom filters is a probabilistic data structure to test set membership , represented by array of m bits and k independent hash functions.
2. Pseudo random Function: a pseudo random function is computationally indistinguishable from a random function. $f : \{0,1\}^n \times \{0,1\}^s \rightarrow \{0,1\}^m$. In the context of this paper a hash function $h_k(w)$ is used as a pseudo random function with k,w as input and the output is an integer between 1 and m where m is the size of bloom filter.
3. Paillier crypto system: Paillier crypto system is an public key crypto system based on decisional composite residuosity assumption (DCRA) is believed to be intractable. It has certain homomorphic properties which makes it suitable for our scheme. It is additively homomorphic and allows multiplication of ciphertext by a constant.
4. Homomorphic Encryption Scheme: Homomorphic encryption is a form of encryption which allows specific types of computations to be carried out on ciphertext and generate an encrypted result which, when decrypted, matches the result of operations performed on the plaintext.
5. Chinese Remainder Theorem: CRT states that for a positive integer with prime factorization $n = p_1^{r_1} \cdots p_k^{r_k}$ we have the isomorphism between a ring and the direct product of its prime power parts: $\mathbf{Z}/n\mathbf{Z} \cong \mathbf{Z}/p_1^{r_1}\mathbf{Z} \times \cdots \times \mathbf{Z}/p_k^{r_k}\mathbf{Z}$
6. Discrete log: If G is a multiplicative cyclic group and g is a generator of G, then from the definition of cyclic groups, we know every element h in G can be written as g^x for some x. The discrete logarithm to the base g of h in the group G is defined to be x . Given a group G, a generator g of the group and an element h of G, to find the discrete logarithm to the base g of h in the group G is considered to be hard.
7. Pohlig Hellman: Pohlig Hellman algorithm is a method to solve discrete log in multiplicative groups with smooth order.

3 Related work and Our Construction

3.1 Secure Indexes

Bloom filters: A bloom filter is a probabilistic data structure used to test set membership. It is represented by an array of m bits and a set of k independent hash functions $h_1, h_2, \dots, h_k$. It represents a set $S = s_1, \dots, s_n$ elements. for each element s_i in the set S, the bit positions corresponding to $h_1(s_i), h_2(s_i) \dots h_k(s_i)$ are set to 1. To test if an element a belongs to the set, it suffices to check if all k bit positions($h_1(a), h_2(a) \dots h_k(a)$) in the bloom filter are set to 1.

But it might so happen that sometimes all the bit positions may have been set by other elements in which case the bloom filter would render an element to be in the set even though it doesnt. This

false positive rate can be reduced by increasing the m or k . to quantify false positive rate of a bloom filter, let n represent the universal set of all unique elements that would ever need to be stored on in the bloom filter. probability that bit i remains 0 after storing all the bits is $(1 - \frac{1}{m})^{kn}$. probability that all hash positions of a specific element are set to 1 (i.e. the false positive rate) is $(1 - (1 - \frac{1}{m})^{kn})^k$. we know that $(1 - \frac{1}{m})^{kn} \approx e^{-\frac{kn}{m}}$. The value of k that minimizes the false positive rate is $k = (\ln 2)(\frac{m}{n})$ with a false positive rate of $(\frac{1}{2})^k$

The scheme consists of the following four algorithms:

Keygen(s): Given a security parameter s , outputs the master key K_{priv}

Trapdoor(K_{priv}, w): Given the master key K_{priv} and word w , it outputs the trapdoor T_w

BuildIndex(D, K_{priv}): Given the master key and document, outputs the index I_D for the document.

SearchIndex(T_w, I_D): Given the trapdoor T_w for word w , and the index I_D for document

The scheme works as follows:

A user wants to upload his documents onto the server in a secure manner yet still wants to selectively retrieve few documents based on the presence of a keyword. Here we consider each document to contain a set of keywords and represent each document with a Bloom Filter. To search for a specific keyword, the server checks set membership of keyword in each document and returns the document which satisfy the condition. Note that even though server may return some documents which do not contain the keyword, the user can always discard the document which does not contain keyword. hence correctness of the scheme holds inspite of false poisitives in bloom filter.

The required secrecy is achieved as follows:

When the user wants to upload a document to the server, the document is broken into a set of keywords and each word is hashed to get k hash values (which are called the trapdoor of the word T_w). The k hash values are appended with the filename and hashed again ($h_{filename}(h_i(w))$) to get k positions in the bloom filter (called the codeword for word w in document d). Here the second level of hashing is required to make sure that the same word does not always hash to same positions in all the document because if it did, it would be trivial for the server to calculate document similarity and this leaks information.

Now when the user wants to retrieve all the documents with the word w , the user finds the trapdoor of the word as above and sends it to the server, the server now performs the Search operation for each document by computing the hash positions $h_{filename}(T_w)$

3.2 Public key setting

The public key scheme also uses Bloom Filters but with special properties called Bloom filters with storage. Here the each position of bloom filter is associated with a buffer instead of a bit as it was in Secure Indexes. To illustrate this concept we first consider bloom filters with storage in a non-cryptographic sense to understand the working and later add encryption scheme to it. Each file(or email) is associated with a unique integer. To store the index of the file, the file is broken into words and for each word, the index is constructed by computing k hash functions specified by the bloom filters to find the respective positions. Here instead of setting the bit at the position to 1, we add the fileid to the buffer at that position. To retrieve all the files corresponding to a word, the user needs to retrieve all the buffers corresponding to bloom filter hashed positions of the word, consider each buffer as potential set of fileids and compute set intersection to get the fileids which contain the required word.

3.2.1 Public Key Encryption Setup

Dan Boneh's paper [2] illustrate a scheme for a user to retrieve all the files containing a keyword in a public key setting. It can be thought of as a mail scenario where a sender wants to send mail to the Receiver which are store on the server. The model considered here is an honest but curious server. We do not want the server to know the details of our emails but we still want to retrieve the emails which contain a know public word. Later we consider a scheme which can retrieve al the emails with a specific keyword which is not public (according to [3]) called PIR (Private Information Retrieval), but it blows up the communication rate which is later reduced significantly by the application of Gentry-Ramzan scheme [4] (which is one of the contributions of this paper apart from implementaion).

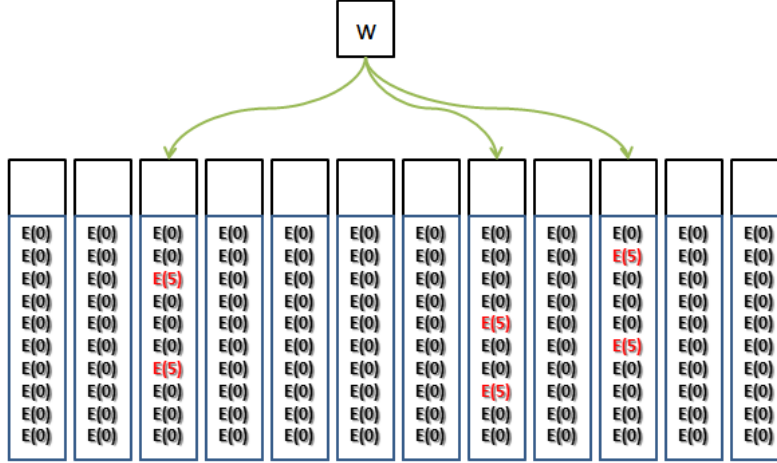
The receiver needs to initialize his mailbox and set all the buffer units in all the buffers with encryption of zero.

E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)
E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)	E(0)

here the size of the bloom filter is determined by the number of unique words that are required to be stored and also on false positive rate. The size of each buffer is fixed beforehand as increasing the buffersize would still reveal some information to the server. this is one of the shortcomings of this scheme as number of files a specific keyword can be associated to needs to be known beforehand.(if the maximum number of files a keyword can be associated to is upperbounded by m , then it the buffer size $m \cdot l$ where l is a constant and the details of why k is required will be explained shortly). This concludes the initMailbox step.

3.2.2 Mail Sending Step

the Sender now wishes to send a mail to the Receiver. The sender would first query the server to get the unique fileid of his to-be-sent-email. the sender would then split the file into words and do the following for each of the word: compute k hashes of the word and find corresponding positions in the bloom filter. the sender now needs to add the fileid to these buffers. As many senders may be sending mails to the same Receiver, the senders repeats these writes to few locations on the buffer and that is the explanation of buffer size to be a multiple of l .



The paper introduces the concept of oblivious modification which is that the sender randomly picks few positions in the buffer and sends fileid encrypted under the Receiver's public key to the server. The server now would update the specified buffer-unit of the specified bloomfilter position by adding up the ciphertexts. This is facilitated by using an additively homomorphic scheme. To make sure that the server does not figure out the bloom filter positions, the sender also sends out encryptions of zeros at random positions for each of the buffers (since it is an additively homomorphic cipher, adding zeros makes no difference). To make sure that while adding a fileid in the bufferunit, it does not add up to an existing fileid, we associate a checksum while adding the fileid which is not additively homomorphic and hence dirty buffer units can easily be recognised and discarded. It is important to choose a large enough buffer size to facilitate correctness. The sender now sends encrypts the file itself with a symmetric cipher such as AES with a random key and adds the AESkey (encrypted under the receiver's public key) to the beginning of the encrypted file. Note here that care must be taken to use appropriate padding while encryption of the key to avoid coppersmith attack of known bits

3.2.3 Retrieval based on Search

As mentioned before, the retrieval process would be considered first for a known keyword, then for PIR query and PIR query with less communication rate.

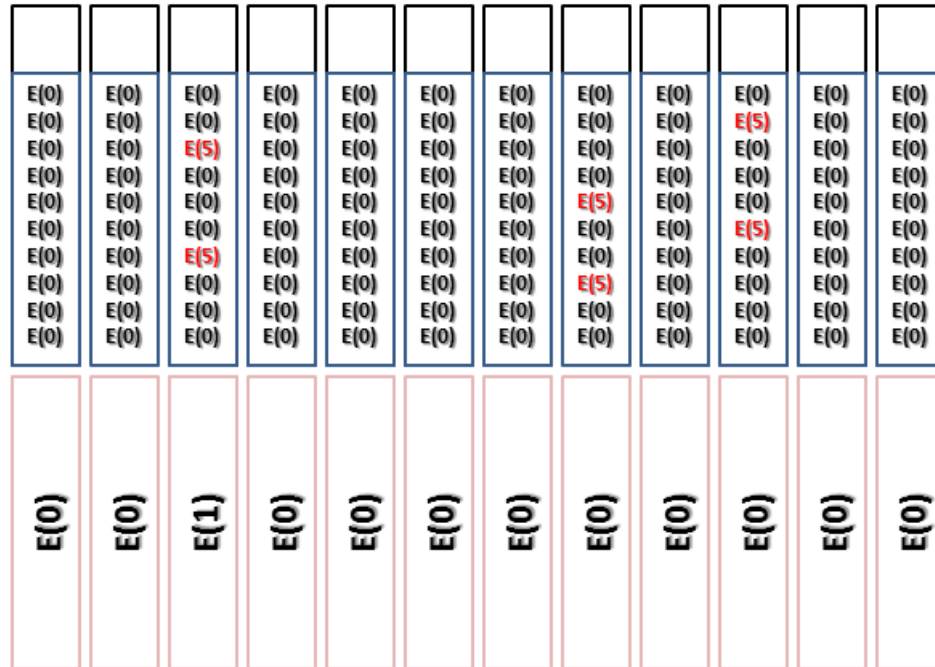
1. keyword Search

The known keyword search is relatively simple. the Receiver wants to retrieve all the emails which have a specific word say 'urgent', the receiver would then compute k hash functions of the bloom filter and find K bloom filter positions and requests the server for these positions. After fetching these buffers, the user decrypts each of the bufferunits in each of the buffers, computes the intersection of the files in each of the buffers and retrieves the files corresponding to those fileids and decrypts the files using the AESKeys which are obtained by decrypting the first few bits.

2. PIR queries

With the known keyword search the server knows the positions of the buffer the receiver requested for and it can be remedied by PIR scheme in [3]. After computing the bloom

filter positions, the receiver retrieves the buffer obviously as follows: the receiver generates another public key whose size is larger than the fixed buffer size. the receiver now sending encryptions of 0 to every positions except for the one buffer positions he cares about. the server would multiply buffer with the encryption ciphers send by the user (in paillier cryptosystem, multiplication by a known constant is feasible). The server then adds all these values which is effectively the value of the encrypted buffer.



The receiver then decrypts the buffer using the bigger key and decrypts the buffer units using the usual smaller key (Note here that in this system the receiver has 2 key pairs - one smaller key which is used to encrypt the fileids by the sender and another bigger key to obviously retrieve selected buffers of the server) and retrieves the set of fileids from each buffer and computes set intersection to obtain the fileids and retrieve the files.

3. Single-Database Private Information Retrieval with Constant Communication Rate

The paper by Gentry Ramzan illustrates a reduced communication rate scheme to obliviously retrieve blocks of bits from a database without revealing the query to the server. the buffer at each position of the bloomfilter can be represented as an integer. The integer is obtained by concatenating all the individual buffer units into one large integer. If the bloom filter has n positions, the first n primes p_i are chosen and raised to an appropriate power e_i so that each $p_i^{e_i}$ is greater than any of the buffer sizes. Now the entire database can be represented by a single integer D which when reduced by $p_i^{e_i}$ yields the value of i^{th} buffer (as shown in figure below). We know that such a D exists by chinese remainder theorem which states that as long as each of $p_i^{e_i}$ as pairwise coprime, such an integer exists mod $\prod_i p_i^{e_i}$.



$$\begin{aligned}
D &= b_1 \mod p_1^{e_1} \\
D &= b_2 \mod p_2^{e_2} \\
&\vdots \\
D &= b_m \mod p_m^{e_m}
\end{aligned}$$

To retrieve document_ids corresponding to a specific word, the user computes the values of each of the hash functions of the bloom filter. Using the results of the hash functions, the user needs to fetch the buffer values at these positions and compute an intersection of ids to get the relevant documents to a specific query.

To fetch the buffer positions i,j,k, The user generates 2 primes p and q of size 4 times the buffer (to avoid coppersmith's attack as mentioned in [4]) such that $p = kp_i^{e_i}p_j^{e_j}p_k^{e_k} + 1$. user then computes $M=pq$ and finds a generator with group order such that $p_i^{e_i}p_j^{e_j}p_k^{e_k}$ divides the order. The user then send M, g to the server.

The server then computes $g^D \mod M$ and sends it to the user.

the user can now compute the values of the buffer by computing discrete log of $(g^D)^{k(q-1)}$ in the sub group generated by $g^{k(q-1)}$. The retrieval scheme is illustrated in the figure.

4 Implementation details

Major development was achieved using python, sage and mongodb.

4.1 Secure Indexes using Bloom Filters

As mentioned above, construction of bloom filters requires Pseudo Random Functions. The pseudo random functions were generated using hmac with sha256. A master key is used to generate k

random keys which are then used as keys for k HMAC. Each position in the bloom filter is one record in mongo db.

4.2 Public Key Encryption with keyword Search

Bloom Filters with storage are implemented with one mongo-collection for each mailbox where each record is one buffer unit. hence the buffer for position i of the bloom filter includes buffer units from $|b|*i$ to $(|b| + 1)*i$. here $|b|$ is the number of buffer units per buffer. For encryption paillier system is used. The ciphers are stored in hex values in the database. While retrieval the values are concatenated and converted to an integer for encryption and further processing.

4.3 Public Key Encryption with PIR queries

for decryption as bigger size paillier key is used to retrieve the entire buffer as described above. Here the entire buffer is concatenated to generate a new integer which should fit the plaintext size. A large prime number(key to paillier cryptosystem) is generated here which takes time but is generated only once and hence acceptable.

4.4 Single-Database Private Information Retrieval with Constant Communication Rate

One of the difficulties faced while implementation of the scheme was to find a generator in the group which has the property that $p_i^{e_i}$ divides the order of the group. Finding multiplicative order is considered as hard as factoring. An attempt was made to iterate through the group elements of M and check for conditions that the candidate g would satisfy $g^{(q-1)!} = 1$ and $g^k! = 1$ which worked for smaller values of integers while using the discrete_log function of sage but checking just the above two conditions failed while using Pohlig Hellman assuming the hidden smooth group . Sage's discrete_log does not accept know factors and hence a custom Pohlig Hellman had to be written . For Pohlig Hellman to work, the extra condition of $g^{k(q-1)p_i^{e_i}}! = 1$ also must be employed in addition to the above 2 conditions. Pohlig Hellman was failing because it could not find the all the digits in the base p_i representation as the multiplicative order divided one of the powers of p_i . But another problem arose as it took a long time to iteratively find a generator satisfying these above properties. A clever solution to this problem was to pick a random element of the group M and check for the above conditions. and this worked best as it always found the generator in atmost 2 iterations of guessing. Using Gentry Ramzan Scheme in the Public key encryption with PIR queries reduces the communication complexity from $m * buffer_size$ to $4 * buffer_size$ which is a significant improvement. Here m is the size of the bloom filter However prime number generation has proven to be a bottle neck for this scheme .

5 Results

Generation of large prime proves to be the bottleneck of the scheme. 16384 bit prime requires 13 minutes to be generated. The above scheme requires an upper bound on the number of files a keyword may belong to. Let λ be the upper bound. If γ is the number of repetitions stored for each keyword, using the paillier cryptosystem, each keyword-file pair saving requires $2 * 1024 * \gamma$ bits to be stored.

Assuming a corpus which contains 1000 unique keywords and a false positive rate of 1 in 64, the bloom filter size would be roughly 8000 and restricting the maximum number of files per

keyword to 100 would require each buffer in the bloom filter to be of size $2 * 1024 * \gamma * 100$ which is 0.256 mb with $\gamma = 10$. The entire bloom filter would be of size 2.048 gb. The secure PIR scheme would require downloading 2 gb of data for each search and Gentry Ramzan PIR scheme would require downloading of 1 mb of data but requires huge computations and generation of prime of roughly $2^{2048000}$ bits.

6 Future work

Although we have used Paillier cryptosystem for additively homomorphic system, using BGN cryptosystem would reduce the communication complexity of sending a mail from $m * bf$ to $\sqrt{m * bf}$. BGN would be much faster with lesser key size and lesser key sizes offer the same level of security in pairing based elliptic curve mechanisms.

While retrieving files at the receivers end using gentry ramzan scheme, the bottle neck of prime generation could be avoided by generating q to be a bunch of 1024 bit primes $q_1, q_2, \dots, q_n$ such that each of these q_i is of the form $2 * t q_i + 1$ so that it does not interfere with the Pohlig Hellman discrete log computation. However finding a suitable p still remains an open problem

Although the Public Key setting offers complete security, it still is not very feasible. The private key setting of bloom filters could be extended to save more information about the words of the file including the part of speech, phrases instead of just words and a comparison can be made about the relevance of the documents retrieved with those of the standard information retrieval schemes.

Another interesting fact about the bloom filters is that a noisy cosine similarity can be calculated between the query and the document and compared against the actual cosine similarity that Information retrieval schemes generally employ.

References

- [1] Kamara, Seny and Papamanthou, Charalampos and Roeder, Tom , *Dynamic Searchable Symmetric Encryption* at Proceedings of the 2012 ACM Conference on Computer and Communications Security, pages 965-976, 2012
- [2] Boneh, Dan and Di Crescenzo, Giovanni and Ostrovsky, Rafail and Persiano, Giuseppe , *Public Key Encryption with Keyword Search* in Advances in Cryptology - EUROCRYPT 2004, Vol 3027, pages 506-522
- [3] Boneh, Dan and Kushilevitz, Eyal and Ostrovsky, Rafail and Skeith, WilliamE., III , *Public Key Encryption that allows PIR Queries* in Advances in Cryptology - CRYPTO 2007, Vol 4622, pages 50-67
- [4] C. Gentry and Z. Ramzan. *Single Database Private Information Retrieval with Constant Communication Rate*. ICALP 2005, LNCS 3580, pp. 803-815, 2005
- [5] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky. *Searchable symmetric encryption: improved definitions and efficient constructions* in ACM Conference on Computer and Communications Security CCS 2006, pages 79-88, 2006
- [6] Eu-Gin-Goh, *Secure Indexes* in Cryptology ePrint Archive, Report 2003, page-216
- [7] Dawn Xiaoding Song; Wagner, D.; Perrig, A., "Practical techniques for searches on encrypted data," Security and Privacy, 2000. S P 2000. Proceedings. 2000 IEEE Symposium on, pages 44-55